

Contents

Forecasting probabilistico di serie temporali con modelli di diffusione	1
Indice	1
0. Sintesi esecutiva	2
1. Introduzione e contributo	2
2. Fondamenti teorici	3
3. Dati e data contract	5
4. La scala di modelli M0–M4	6
5. Metodologia di valutazione: i quattro pilastri	7
6. Esperimenti e risultati	9
7. Discussione	13
8. Limiti e minacce alla validità	13
9. Conclusioni e lavoro futuro	14
10. Riproducibilità	14
Appendice A — Glossario delle metriche	15
Appendice B — Mappa del repository	15
Appendice C — Riferimenti	16

Forecasting probabilistico di serie temporali con modelli di diffusione

Report scientifico — progetto d’esame di Probabilistic Machine Learning (PML) Università degli Studi di Trieste · Prof. Luca Bortolussi · gruppo di 3.

Domanda di ricerca. Un diffusion model può produrre forecast probabilistici di una serie temporale — una distribuzione di futuri plausibili invece di un singolo numero — più accurati, meglio calibrati o più informativi delle baseline classiche e di deep-learning, e a quale costo computazionale e con quale valore economico?

Nota di lettura. Il report è scritto in italiano con i termini tecnici in inglese (CRPS, coverage, forward process, ...), coerentemente con il resto del repository. Ogni numero citato è reale, proviene da `results/registry.csv` (singola sorgente di verità) ed è qui arrotondato senza perdita d’informazione. Le tabelle di confronto non sono scritte a mano: sono generate da `experiments/make_tables.py`. Dove un esperimento non è ancora stato eseguito, lo dichiariamo esplicitamente: il valore scientifico del progetto è la sua onestà sperimentale, non un risultato gonfiato.

Indice

0. Sintesi esecutiva
1. Introduzione e contributo
2. Fondamenti teorici
3. Dati e data contract
4. La scala di modelli M0–M4
5. Metodologia di valutazione: i quattro pilastri
6. Esperimenti e risultati
7. Discussione

- 8. Limiti e minacce alla validità
 - 9. Conclusioni e lavoro futuro
 - 10. Riproducibilità
 - Appendice A — Glossario delle metriche
 - Appendice B — Mappa del repository
 - Appendice C — Riferimenti
-

0. Sintesi esecutiva

Inquadriamo il forecasting come l'apprendimento della distribuzione condizionata $p(\text{futuro} \mid \text{passato})$ e la realizziamo con un conditional diffusion model. Il contributo originale rispetto al programma del corso — che culmina sui diffusion models per la generazione incondizionata — è estendere quell'ultimo capitolo dalla generazione incondizionata al forecasting condizionato, e discutere esplicitamente quale incertezza il modello cattura (aleatoria) e quale no (epistemica).

Costruiamo una scala di modelli di severità crescente — M0 seasonal-naive, M1 ARIMA, M2 DeepAR, M3 TimeGrad (diffusione condizionale autoregressiva, il centerpiece), M4 TimeDiff (diffusione non-autoregressiva) — e la valutiamo su quattro pilastri: accuratezza puntuale (MAE/RMSE/MASE), qualità probabilistica (CRPS, coverage, calibrazione), costo, e valore economico (denaro risparmiato programmando una batteria contro un prezzo a fasce). Il confronto è reso onesto da un data contract che garantisce per costruzione che tutti i modelli vedano gli stessi split, lo stesso scaling e le stesse finestre.

Risultato chiave (Electricity, CRPS — la metrica probabilistica di riferimento). La barra del seasonal-naive è 160.5 e nessun modello addestrato la batte: DeepAR 253.7, TimeGrad 241.6 (meglio di DeepAR ma non del naive), TimeDiff 287.3, ARIMA 867.5. È un finding onesto e non-trionfalistico: su un segnale fortemente stagionale una baseline semplice è un avversario serio, e la maggiore ricchezza distribuzionale della diffusione non ripaga il suo costo di sampling, qui. I due diffusion model lo mostrano da estremi opposti — M3 TimeGrad (autoregressivo) è il più calibrato ma lento (~4h 19min di sampling), mentre M4 TimeDiff (non-autoregressivo) campiona in ~41min ed ha l'errore puntuale più basso tra i deep, ma nella forma x_0 la sua predittiva collassa (coverage ≈ 0 , CRPS $\approx$ MAE).

La scoperta più didattica. L'ablazione che predice il rumore ϵ (DDPM standard) invece del segnale pulito x_0 non ricalibra TimeDiff: lo ribalta nell'estremo opposto (coverage ≈ 1 , intervalli $\sim 40\times$ più larghi, MASE 4.0). x_0 sotto-disperde, ϵ sovra-disperde: la parametrizzazione è una tensione reale, non uno switch, e calibrare davvero richiede più di un cambio di target. Il modello meglio calibrato dell'intera scala è DeepAR, non una diffusione.

1. Introduzione e contributo

1.1 Forecasting come distribuzione condizionata

La maggior parte dei forecast restituisce un singolo numero — “la domanda elettrica di domani sarà 100”. Ma il futuro è incerto, e un numero solo nasconde del tutto quell'incertezza. Un forecast più utile dice: “ecco i domani plausibili, e quanto è probabile ciascuno” — un'intera distribuzione di futuri. For-

malmente, data una serie multivariata $x_{1:L} \in \mathbb{R}^{D \times L}$, un contesto di H passi $c = x_{t-H+1:t}$ e un orizzonte di y passi $y = x_{t+1:t+}$, il compito è stimare e campionare $p(x_{t+1:t+} | x_{t-H+1:t})$, ossia $p(\text{futuro} | \text{passato})$.

1.2 Il contributo: dal generativo incondizionato al forecasting condizionato

Il corso PML termina sui diffusion models come modelli generativi incondizionati (cap. 11 delle dispense). L'originalità del progetto è estendere quel capitolo finale alla generazione condizionata: rendere il denoiser dipendente dal passato, $\epsilon(x_t, t, c)$ con $c = \text{Encoder}(x_{\text{passato}})$. Tutto il resto è la §11.2 delle dispense, applicata al forecasting. Su questo asse il progetto è genuinamente originale rispetto ai contenuti del corso, che il conditional non lo costruisce mai.

1.3 La tesi (deliberatamente non-trionfalista)

Per il forecasting la domanda giusta non è cosa accadrà, ma cosa potrebbe accadere e quanto ne siamo sicuri. I diffusion model rispondono nativamente alla seconda domanda — generano campioni del futuro — ma il premio è la qualità dell'incertezza, non un MAE più basso, e arriva a un costo di sampling che misuriamo e tuniamo. La tesi è falsificabile: se la diffusione perde, è comunque un risultato. Come si vedrà, su Electricity la diffusione non batte il seasonal-naive sul CRPS — e questo, lungi dall'essere una sconfitta, è il cuore onesto del lavoro.

2. Fondamenti teorici

2.1 Incertezza aleatoria vs epistemica

Il corso distingue due incertezze (dispense §1.1.1): aleatoria, irriducibile, il rumore intrinseco dei dati (la dispersione dei futuri plausibili); ed epistemica, riducibile, l'incertezza su modello/parametri. I modelli bayesiani (Ch. 6), le BNN (§10.6) e i GP (Ch. 12) catturano quella epistemica mettendo distribuzioni sui parametri.

Un diffusion forecaster con pesi stimati puntualmente cattura l'incertezza aleatoria (la dispersione dei futuri plausibili) ma non quella epistemica (è "sicuro" dei propri parametri).

È una precisazione load-bearing per l'orale: quale incertezza stiamo quantificando, e cosa ci manca? La risposta naturale per il lavoro futuro è una diffusione bayesiana/ensemble.

2.2 Il modello di diffusione (DDPM)

Forward process (fisso). Si corrompe progressivamente il dato pulito x_0 con T passi di rumore gaussiano:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1-\beta_t} \cdot x_{t-1}, \beta_t I), \quad \beta_t = 1 - \alpha_t, \quad \alpha_t = \prod_{i=1}^t \alpha_i.$$

Forma chiusa (reparameterization trick). Componendo i passi si salta direttamente a un qualunque livello di rumore:

$$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1-\alpha_t} \cdot \epsilon, \quad \epsilon \sim N(0, I) \quad q(x_t | x_0) = N(\sqrt{\alpha_t} x_0, (1-\alpha_t) I).$$

Per $t \rightarrow T$, $\alpha_t \rightarrow 0$ e $x_T \rightarrow N(0, I)$: rumore puro.

Reverse process (appreso). Si impara a invertire la corruzione, $q_{\phi}(x_{t-1} | x_t) = N(\mu_{\phi}(x_t, t), \Sigma_{\phi}(x_t, t))$, addestrando per massima verosimiglianza (equivalentemente minimizzando un ELBO / la KL $KL(p(x_{0:T}) || q_{\phi}(x_{0:T}))$).

ϵ -prediction (la loss semplice di Ho et al. 2020). Riparametrizzando la rete perché predica il rumore ϵ iniettato invece del segnale x_0 , l'obiettivo collassa in un MSE:

$$L(\phi) = E_{t, x_0} \| \mu_{\phi}(\sqrt{t} \cdot x_0 + \sqrt{1-t} \cdot \epsilon, t) \|^2 \quad (\text{Algoritmo 1}).$$

Sampling (Algoritmo 2). Si parte da $x_T \sim N(0, I)$ e si percorre la catena inversa:

$$x_{t-1} = (1/\sqrt{t}) (\sqrt{t} x_t - (1-\sqrt{t})/\sqrt{1-t} \cdot \mu_{\phi}(x_t, t)) + \sqrt{t} \cdot z, \quad z \sim N(0, I).$$

β -schedule. La sequenza $\beta_1 \dots \beta_T$ (lineare o coseno) governa quanto rumore si aggiunge per passo; è un'ablazione naturale (M3 usa linear; M4 usa cosine).

2.3 Il salto condizionale

Il forecasting si ottiene rendendo il denoiser condizionato sul passato:

$$\mu_{\phi}(x_t, t, c), \quad c = \text{Encoder}(x_{\text{passato}}).$$

La letteratura chiama questo conditioning source = serie storica e condition integration = come c entra nella rete. I due modelli di diffusione del progetto realizzano il condizionamento in modo opposto (§4.4–4.5): TimeGrad lo fa in modo autoregressivo, un passo alla volta, guidato dallo stato nascosto di un RNN; TimeDiff lo fa in modo non-autoregressivo, denoising l'intero blocco futuro x_D in una volta a partire da un'inizializzazione lineare.

2.4 Mappatura al corso

Ogni componente è ancorato alle dispense di Bortolussi (l'offset PDF è +10 rispetto alla pagina stampata):

Componente	Dispense
Forecasting come $p(\text{futuro} \text{passato})$	§1.2 Generative Modelling
Perché una distribuzione, non un punto (aleatoria vs epistemica)	§1.1.1
Forward process / forma chiusa di noising	§11.2.1
Reverse / processo generativo	§11.2.2
Obiettivo di training, loss ottimizzata, Alg. 1	§11.2.2–11.2.3
Sampling (Alg. 2), ancestral sampling	§11.2.3 / §3.3.1
Macchinario ELBO (condiviso col VAE)	§9.2 / §11.1
Reparameterization trick	§10.4.1
Vista score-based / Langevin	§11.2.5
DeepAR (distribuzione predittiva)	§1.2 / §6.5
ARIMA/ETS (state-space $\approx$ HMM; predittiva gaussiana)	Ch. 5 / Ch. 6
CRPS / proper scoring rules	§2.3
Valore economico (teoria della decisione, expected loss)	§2.3 / §6.5

Il one-liner per l'orale: il progetto estende la §11.2 (diffusione) dalla generazione incondizionata al forecasting condizionato.

3. Dati e data contract

3.1 I due dataset

Non sono due progetti separati ma due iterazioni dello stesso identico codice: cambia solo il file di config.

Campo	Exchange (sandbox)	Electricity (primario)
Ruolo	iterazione / get-it-green-first	la headline
Dominio	8 tassi di cambio vs USD	321 serie di carico elettrico
Sorgente	LSTNet exchange_rate.txt.gz	LSTNet electricity.txt.gz
D (canali)	8	321
Frequenza	giornaliera	oraria
context H	60	168 (una settimana)
orizzonte	30	24 (un giorno)
stagionalità m	1 (random walk)	24 (ciclo giornaliero)
split	0.7 / 0.1 / 0.2	0.7 / 0.1 / 0.2 (identico)
seed	42	42
finestre di test	1 430	5 071

Perché Exchange prima. È piccolo (D=8, pochi minuti per l'intera scala). Lo usiamo per portare tutta la pipeline in verde — download, split, scaling, finestratura, training, sampling, metriche, registry — prima di spendere ore sul dataset grande. Ha pagato: i bug strutturali (il reload del checkpoint DeepAR sotto PyTorch ≥ 2.6 ; il bug di frequenza di GluonTS 0.13) sono emersi su Exchange in secondi, non dopo ore.

Il ruolo scientifico di Exchange (controllo). Exchange $\approx$ random walk (autocorrelazione a lag-1 ~ 0.999 ; vedi docs/EDA_EXCHANGE.md): il miglior predittore è la persistenza (“domani = oggi”). È un controllo: qui i modelli deep non dovrebbero battere il naive — e infatti non lo fanno (§6.3). Questo conferma che la pipeline è onesta, non gonfia i risultati: “la diffusione non fa miracoli su una martingala”.

3.2 Il data contract (il cuore dell'onestà sperimentale)

Un unico loader restituisce gli split (train, val, test) più un dizionario di metadati (D, freq, H, , scaler); ogni modello consuma lo stesso oggetto, quindi il confronto è equo per costruzione — nessuno può barare su un dataset. Tre garanzie:

- Split temporale anti-leakage. train | val | test contigui in ordine di tempo (70/10/20), mai mescolati; il test è la porzione più recente.
- Scaling fit-solo-su-train. Z-score per-canale, parametri stimati solo sul train, applicati a val/test. Lo scaling leakage è il bug silenzioso #1: il contratto lo previene.
- Finestratura interna. Finestre scorrevoli (H,) dentro ciascuno split; nessuna finestra attraversa il confine.

Tutte le metriche sono calcolate sulla scala originale (de-scalata) dei dati, così i numeri sono interpretabili. La promessa operativa è “cambiare dataset = cambiare un YAML”: nessun numero magico nel codice.

3.3 Rappresentazione del forecast (apples-to-apples)

Perché il confronto sia equo, ogni modello produce sia un punto sia una distribuzione:

Modello	Punto	Distribuzione
M0 seasonal-naive	il valore stagionale	banda da residual bootstrap
M1 ARIMA/ETS	media predittiva	predittiva gaussiana analitica ($\pm z \cdot \sigma$)
M2 DeepAR	media/mediana dei campioni	S traiettorie campionate
M3 TimeGrad	media/mediana dei campioni	S traiettorie campionate
M4 TimeDiff	media/mediana dei campioni	S traiettorie campionate

Per i modelli a campioni usiamo $S = 100$ traiettorie per finestra.

4. La scala di modelli M0–M4

Ogni gradino è un avversario più forte del precedente. M0–M3 sono il nucleo richiesto; il quarto gradino, previsto dal piano come opzionale (originariamente ipotizzato come CSDI/GP), è stato realizzato come M4 TimeDiff — una scelta che ha pagato in robustezza (torch puro, niente pin fragili) e ha prodotto la scoperta più didattica del progetto (§6.4).

4.1 M0 — Seasonal-naive (l’ancora di onestà)

“Stupido ma essenziale”: il forecast è il valore osservato m passi prima. È la barra da battere: un seasonal-naive forte è notoriamente difficile da superare su segnali stagionali. La sua versione probabilistica usa un residual bootstrap per ottenere una banda (così ha un CRPS e una coverage, e può persino alimentare il piano stocastico di E6).

4.2 M1 — ARIMA (la tradizione statistica)

Auto-ARIMA per-canale (statsforecast di Nixtla), con intervalli predittivi gaussiani analitici ($\pm z \cdot \sigma$). Mappa allo state-space lineare-gaussiano ($\approx$ HMM, Ch. 5) + predittiva gaussiana (Ch. 6). Apre la “scatola”: dove la struttura gaussiana/lineare si rompe (code, non-linearità), ARIMA paga.

4.3 M2 — DeepAR (la baseline deep equa)

RNN autoregressivo con verosimiglianza parametrica (Student-t/gaussiana) per passo; la distribuzione predittiva emerge dal roll-out Monte-Carlo della catena autoregressiva (GluonTS). È la baseline deep equa: già distribuzionale, così la diffusione non viene confrontata con una rete deterministica. “Stesso spirito generativo della diffusione, meccanismo diverso.”

4.4 M3 — TimeGrad (il centerpiece: diffusione condizionale autoregressiva)

Architettura: un RNN codifica la storia in uno stato nascosto h_t (il condizionamento c), e a ogni passo del forecast un DDPM condizionale fa il denoising di un campione del prossimo vettore multivariato dato h_t . È esattamente la §11.2 resa condizionale.

Implementazione: PyTorchTS TimeGradEstimator (dell'autore di TimeGrad, sopra GluonTS — la versione canonica e riproducibile). Predice ϵ (loss DDPM). Iperparametri Electricity: `diff_steps=100, =linear, RNN GRU num_cells=64, layers=2, epochs=50, S=100, seed=42`. È autoregressivo sull'orizzonte: passi RNN $\times$ catena di denoising $\rightarrow$ il modello più lento della scala (sampling ~4h 19min su Electricity).

4.5 M4 — TimeDiff (diffusione condizionale non-autoregressiva) + ablazione ϵ

A differenza di M3 (denoising un passo alla volta, guidato da RNN), M4 (Shen & Kwok, ICML 2023) denoising l'intero blocco futuro $\times D$ in una volta:

- target x_0 (loss MSE sul segnale pulito), non ϵ ;
- inizializzazione lineare/AR: un `Linear(H  $\rightarrow$  )` per canale produce x_{ar} come punto di partenza informato;
- future-mixup in training (in inferenza si usa solo x_{ar} , niente leakage);
- backbone conv gated stile DiffWave;
- torch puro: niente GluonTS/PyTorchTS, gira sul torch “stock” di Colab (critical path più corto e robusto del pin di M3).

Payoff atteso e confermato: sampling molto più veloce di M3 (una sola catena inversa, non passi RNN) — ~41min contro ~4h 19min. Iperparametri Electricity: `hidden=64, blocks=4, =cosine, mixup=0.5, sample_steps=100, =1.0, diff_steps=100, epochs=50, S=100, seed=42`.

Ablazione M4 ϵ . Una variante identica che predice ϵ invece di x_0 (`--param eps`), per testare se la parametrizzazione del rumore ricalibra il modello. L'esito (§6.4) è la scoperta centrale del lavoro.

4.6 toy DDPM (artefatto di comprensione)

Un DDPM condizionale from-scratch in ~150 righe (denoiser MLP/1D-CNN $_{(x_t, t, c)}$, $T=100$, β lineare; training con la loss §11.2.3, sampling con l'Alg. 2) su una serie univariata. Serve come prova più pulita di comprensione e ad alimentare E3 senza attriti di libreria. Stato: in lavorazione (abbozzato per E3).

5. Metodologia di valutazione: i quattro pilastri

Un forecast probabilistico va giudicato su due famiglie di metriche: la distanza dalla verità (punto) e se la distribuzione è ben calibrata e netta (probabilistico). Le formule che seguono sono quelle effettivamente implementate in `src/eval/metrics.py`.

5.1 Pilastro 1 — Accuratezza puntuale

Con y_{true} , $point \in \mathbb{R}^{N \times D}$:

$$MAE = \text{mean } |y - \hat{y}|, \quad RMSE = \sqrt{\text{mean } (y - \hat{y})^2}.$$

MASE (Mean Absolute Scaled Error) — scala-libera, comparabile tra serie di magnitudini diverse: la MAE divisa per la MAE in-sample del seasonal-naive sul train,

$scale_d = \text{mean}_t |x_{\{t,d\}} - x_{\{t-m, d\}}| \quad (\text{sul train})$, $MASE = \text{mean} (|y - \hat{y}| / scale)$

MASE < 1 batte il naive sul suo stesso metro; MASE = 1 lo pareggia. MASE e CRPS dipendono da scala e denominatore del dataset: sono comparabili solo entro lo stesso dataset.

5.2 Pilastro 2 — Qualità probabilistica

CRPS (Continuous Ranked Probability Score) — la metrica di riferimento; generalizza la MAE alle distribuzioni e per una predittiva degenerare (tutti i campioni uguali) si riduce a $|x-y|$. Usiamo lo stimatore fair / quasi-non-distorto da ensemble (Zamo & Naveau 2018), in $O(S \log S)$:

$CRPS = (1/S) \sum_i |x_i - y| - 1/(S(S-1)) \sum_i (2i - S - 1) x_{(i)}$,

con $x_{(i)}$ i campioni ordinati. È proper (Gneiting & Raftery 2007) e premia forecast calibrati (massa nel posto giusto) e netti (non inutilmente larghi).

Coverage e width. Per un intervallo centrale a livello `level` (50% e 90%): la coverage è la frazione di verità che cade dentro (`target = level`; sopra $\Rightarrow$ under-confident/troppo largo, sotto $\Rightarrow$ over-confident/troppo stretto); la width è l'ampiezza media (la sharpness). Il goal è l'intervallo più stretto che raggiunge la coverage nominale.

Pinball loss. Media della quantile loss $_{-q}$ su una griglia di q ; poiché $CRPS = 2 \int_{-\infty}^{\infty} _{-q} dq$, vale $2 \times \text{pinball}$ CRPS — un cross-check che le due metriche probabilistiche raccontino la stessa storia.

5.3 CRPS-sum — la metrica pubblicata (E0)

La letteratura (TimeGrad, CSDI, ScoreGrad) riporta su `electricity_nips` la CRPS-sum multivariata, calcolata esattamente come `gluonts.evaluation.MultivariateEvaluator` con `target_agg_funcs={"sum": np.sum}`:

$CRPS_sum = \text{mean}_q [(\sum_t 2 \cdot _{-q}(y_t^{\wedge \Sigma}) , q\text{-esimo quantile di } F_t^{\wedge \Sigma})) / \sum_t |y_t^{\wedge \Sigma}|]$,

dove $y_t^{\wedge \Sigma} = \sum_d y_{\{t,d\}}$ è la somma sui canali (si valuta il forecast congiunto dell'aggregato) e q scorre i decili 0.1...0.9. La somma-sui-canali e la normalizzazione per $\sum |y^{\wedge \Sigma}|$ la rendono scale-free (Electricity ~ 0.02).

□ CRPS-sum e CRPS non sono sulla stessa scala ($\sim 10^{-2}$ vs $\sim 10^2$) e non vanno mai confrontati direttamente. Sono due metriche diverse: CRPS per-posizione (la nostra colonna headline) e CRPS-sum normalizzato (per il confronto col paper).

La CRPS-sum è opt-in (`--crps-sum`): di default ogni run produce il dict identico a prima (così nessun numero già in banca viene toccato), perché il numero è confrontabile con la letteratura solo sotto il protocollo pubblicato (`split electricity_nips + finestre rolling`) — vedi E0 (§6.1).

5.4 Pilastro 3 — Costo

Wall-clock di training e di inferenza (tempo per S traiettorie/finestra), numero di parametri, e la curva diffusion-specifica qualità-vs-passi di denoising $T \rightarrow E3$, dove il famoso limite del sampling lento della diffusione diventa un risultato misurato.

5.5 Pilastro 4 — Valore economico (E6)

Un forecast conta solo se cambia una decisione. Usiamo ogni forecast per programmare una batteria (caricare quando l'energia costa poco, scaricare quando costa molto) contro un prezzo time-of-use, poi prezziamo il risultato. Per ogni modello costruiamo due piani:

- un LP deterministico (forecast puntuale → una schedule);
- un LP stocastico che minimizza la bolletta attesa sulle S traiettorie campionate (sample-average approximation, SAA) — solo i modelli probabilistici possono farlo.

Ogni schedule è applicata al futuro vero per leggere la bolletta realizzata; si riportano il denaro risparmiato vs naive (soffitto) e la frazione del risparmio ottenibile dall'oracolo (previsione perfetta, pavimento). Il legame coi pilastri: le decisioni ottime di storage usano un quantile della predittiva (struttura newsvendor), e la CRPS è il regret decisionale medio su tutti i rapporti di costo — quindi un forecast meglio calibrato dovrebbe far risparmiare di più. E6 testa se lo fa, su denaro vero. È un modulo bolt-on (`src/eval/economic.py`) che gira sopra i forecast già prodotti da E1 — nessun training aggiuntivo.

5.6 Igiene statistica e principio di equità

Stessi split/scaling/H/finestre per tutti; metrica puntuale sempre dalla media predittiva; niente tuning asimmetrico (si tunano entrambi o nessuno, e lo si dichiara); niente scelte post-hoc (orizzonte/seed/soglia fissati prima). Regola di significatività: se due modelli sono entro una deviazione standard, non si dichiara un vincitore. (Onestà: il nostro protocollo effettivo usa un solo seed, 42 — vedi §8.)

6. Esperimenti e risultati

Ogni esperimento è numerato, falsificabile e produce un artefatto. Un buon esperimento può deludere la nostra tesi, e resta comunque un risultato.

6.1 E0 — reproduce-gate (electricity_nips + CRPS-sum)

Ipotesi. La pipeline riproduce un risultato pubblicato entro lo stesso ordine di grandezza. Motivo. Tutti i numeri headline di Electricity girano sul file grezzo LSTNet con il nostro split 70/10/20, mentre i CRPS-sum dei paper sono riportati su `electricity_nips` (370 serie, confine train/test ufficiale, test a 7 finestre rolling con `prediction_length=24`). Confrontare il nostro CRPS (scala $\sim 10^2$) col loro CRPS-sum ($\sim 10^{-2}$) sarebbe un errore di categoria. E0 isola una sola variabile: stesso modello (M3 TimeGrad, seed 42), $H=168$ e $\tau=24$ identici alla config LSTNet, cambia solo la sorgente dati e lo split.

Stato. Prep completo (config `configs/data_electricity_nips.yaml`, loader `src/data/electricity_nips.py`, metrica CRPS-sum implementata e testata, flag `--crps-sum` sul runner, cella Colab P2.6, doc `docs/E0_REPRODUCE_GATE.md`). La metrica è verificata da test che la pinnano contro una reference GluonTS-style hand-rolled, contro l'accumulatore streaming, e contro il forecast perfetto → 0. Il run su GPU resta da eseguire (vive solo nell'ambiente pesante gluonts, su Colab). Numeri di riferimento attesi ($\sim 10^{-2}$): TimeGrad ~ 0.021 (Rasul 2021), CSDI ~ 0.017 – 0.018 (Tashiro 2021), GP-Copula ~ 0.024 (Salinas 2019).

Come si interpreterà. Se il nostro CRPS-sum E0 si avvicinerà a ~ 0.02 , gran parte della distanza headline dai numeri pubblicati era lo split (321 vs 370 serie, sliding vs rolling), non un difetto del

modello; se resterà molto più alto, la differenza è imputabile a budget/tuning/seed (1 seed, 50 epoche, niente tuning esteso) — anch'esso un limite onesto. In entrambi i casi E0 trasforma un limite vago (“split diverso”) in un numero.

6.2 E1 — confronto principale (Electricity, il banco di prova)

Setup. `test split · n_windows=5071 · H=168 · =24 · D=321 · S=100 · seed=42 · m=24.`
Generato da `experiments/make_tables.py` da `results/registry.csv`.

Qualità:

Modello	MASE	CRPS	pinball	cov50	cov90	MAE	RMSE
M0 seasonal- naive	1.003	160.5	84.6	0.497	0.885	203.0	1 759.7
M1 ARIMA	3.664	867.5	459.0	0.592	0.930	1 131.9	12 870.6
M2 DeepAR	1.830	253.7	133.9	0.466	0.831	356.0	2 448.8
M3 TimeGrad	1.391	241.6	126.6	0.279	0.647	312.8	3 025.3
M4 TimeDiff (x0)	1.399	287.3	143.8	0.003	0.008	288.5	2 551.5
M4ε TimeDiff (ε)	4.017	1 376.3	729.0	0.998	1.000	1 044.8	9 370.4

Grassetto = miglior modello per colonna; la coverage premia la vicinanza al nominale (0.50 / 0.90), non il valore più alto.

Costo & setup:

Modello	fit	predict	epochs	diff_steps	piattaforma
M0 seasonal- naive	—	~58.7 min	—	—	macOS (locale)
M1 ARIMA	~29.3 min	~2h 42min	—	—	macOS (locale)
M2 DeepAR	~10.1 min	~82.2 min	50	—	Linux (Colab)
M3 TimeGrad	~38.9 min	~4h 19min	50	100	Linux (Colab L4)
M4 TimeDiff (x0)	48 s	~41.4 min	50	100	Linux (Colab)
M4ε TimeDiff (ε)	43 s	~40.7 min	50	100	Linux (Colab)

Lettura. La barra sul CRPS è 160.5 (il seasonal-naive M0). Nessun modello addestrato la batte: DeepAR si ferma a 253.7, TimeGrad a 241.6 (meglio di DeepAR — la diffusione qualcosa aggiunge

sul probabilistico — ma non abbastanza da scalzare il naive, e a un costo di sampling ~4h 19min), TimeDiff 287.3, ARIMA 867.5 fuori gioco. È lo Scenario A non-trionfalista: il valore del lavoro è il confronto controllato, non una vittoria del modello.

Tre osservazioni scientifiche:

1. TimeGrad è sovra-confidente. cov50 0.279 e cov90 0.647 (contro 0.50 e 0.90 nominali): le bande sono troppo strette. Migliora il CRPS rispetto a DeepAR ma a scapito della copertura.
2. DeepAR è il meglio calibrato della scala (cov50 0.466, cov90 0.831, i più vicini al nominale tra i modelli addestrati). Un risultato controintuitivo e onesto: il meglio calibrato non è una diffusione.
3. M4 mostra i due fallimenti opposti della parametrizzazione (§6.4): x_0 collassa (cov ≈ 0), ε esplode (cov ≈ 1).

6.3 Exchange — il controllo (la pipeline non gonfia)

Setup. test · n_windows=1430 · H=60 · =30 · D=8 · S=100 · seed=42 · m=1.

Modello	MASE	CRPS	pinball	cov50	cov90	MAE	RMSE
M0 seasonal- naive	4.526	0.00718	0.00379	0.444	0.862	0.00981	0.01659
M1 ARIMA	4.525	0.00725	0.00383	0.633	0.932	0.00982	0.01658
M2 DeepAR	7.407	0.01025	0.00542	0.392	0.828	0.01413	0.02185
M3 TimeGrad	7.478	0.01134	0.00596	0.384	0.729	0.01504	0.02383

Su Exchange (random walk) naive $\approx$ ARIMA vincono e i modelli deep peggiorano (MASE ~7.4–7.5 contro ~4.5): è il risultato atteso e valida che la pipeline non “gonfia”. Nota: ARIMA è qui il più calibrato (cov90 0.932), TimeGrad sotto-copre (cov90 0.729).

□ I CRPS di Exchange (~0.01) ed Electricity (~160) sono su scale diverse e incomparabili in valore assoluto. Solo i ranking entro uno stesso dataset sono significativi.

6.4 La scoperta: collasso $x_0 \rightarrow \varepsilon$ ribalta (il pezzo didattico più forte)

Il quinto gradino è stato bancato in due parametrizzazioni reali, e l'esito è più ricco dell'ipotesi di partenza.

M4 x_0 — il collasso di varianza. cov ≈ 0 non è un bug del sampler (la matematica DDIM è corretta). È il collasso di varianza della x_0 -prediction: la rete impara a predire un x_0 quasi costante, appoggiandosi al contesto e all'init lineare x_{ar} e ignorando il rumore x_t . L'unica dispersione residua nella catena inversa è quella dell'ultimo passo, $1 - \beta_0 \approx 6 \cdot 10^{-2} \rightarrow \text{std} \approx 0$ in spazio standardizzato. La future-mixup aggrava (insegna a copiare il condizionamento, deterministico in inferenza). Spia indipendente: CRPS 287.3 $\approx$ MAE 288.5 — per una predittiva a massa puntiforme il CRPS coincide con la MAE. Verificato con simulazione numerica della ricorsione di varianza.

M4 ϵ — ϵ non calibra, ribalta. Tesi iniziale: predire il rumore ϵ (DDPM standard, Ho 2020) invece del segnale pulito x_0 controlla esplicitamente la dispersione iniettata → la calibrazione dovrebbe tornare. Esito reale: la tesi è smentita — ϵ non ricalibra, ribalta.

metrica	M4 x_0	M4 ϵ	nominale
cov50 / cov90	0.003 / 0.008	0.998 / 1.000	0.50 / 0.90
width50 / width90	$\approx 2.8 / 6.6$	7 686 / 11 359	—
MASE	1.399	4.017	—
CRPS	287.3	1 376.3	—
MAE	288.5	1 044.8	—

Diagnosi (non è un bug). Il rapporto RMSE/MAE resta uniforme (ϵ 8.97 $\approx$ x_0 8.85 $\approx$ naive 8.67) → è sovra-dispersione uniforme su tutte le finestre, non qualche finestra esplosa; le formule sono i DDPM da manuale (x_0 -recovery $(x_t - \sqrt{(1-\alpha_t)})/\sqrt{\alpha_t}$, target ϵ = rumore iniettato); nessun NaN. Meccanismo: senza ancoraggio autoregressivo, il blocco non-AR sparge la varianza iniettata da ϵ su tutte le x_D celle senza ricompilarla → varianza fuori controllo. È lo specchio esatto del collasso x_0 : i due estremi opposti della stessa tensione di parametrizzazione.

L'arco didattico (più forte dell'ipotesi). x_0 sotto-disperde (cov ≈ 0) → predire ϵ non calibra ma ribalta in sovra-dispersione (cov ≈ 1) → la parametrizzazione è una tensione reale, non uno switch; x_0 è il male minore (la scelta del paper TimeDiff); calibrare davvero richiede di più (una _ appresa, o una correzione conformal); e il meglio calibrato della scala resta DeepAR, non una diffusione.

6.5 E6 — valore economico (battery dispatch)

Stato. Modulo costruito e testato (src/eval/economic.py, 10 test verdi). Demo M0 in locale: dispatch senza export, batteria 4h @ 0.50-media con $\eta=0.9$, tariffa TOU picco/spalla/notte = 0.3/0.15/0.08, su 6 canali (top-variance) $\times$ 120 finestre, =24.

	naive	oracolo	det	risparmio det	% del max	sto	Δ dist
TOTALE	913 490.6	854 504.9	860 755.1	52 735.6	89%	861 053.5	-298.4

Lettura. Programmare la batteria sul forecast naive recupera l'89% del risparmio massimo ottenibile dall'oracolo (previsione perfetta) — il forecast, anche semplice, vale denaro vero. La colonna Δ dist = bolletta(det) - bolletta(sto) misura il valore della distribuzione: per M0 è leggermente negativo (-298.4), perché la "distribuzione" del naive è solo un residual bootstrap, povera di struttura. È un risultato onesto e atteso: il payoff della distribuzione emergerà col confronto cross-model (CRPS-vs-€) quando saranno disponibili gli array di forecast di M1/M2/M3 — l'infrastruttura per quel confronto è pronta.

6.6 E2 / E3 / E4 — stato

- E2 (horizon sweep) — in corso. Infrastruttura pronta (run_horizon_sweep.py, plot_sweeps.py) + leg locale M0; i leg deep (M2/M3) girano su Colab. Ipotesi: il vantaggio della diffusione cresce con .

- E3 (denoising-steps vs costo/qualità) — in corso. Curva qualità-vs-T via toy DDPM (NumPy, locale) + leg M3 reale su Colab. Ipotesi: un ginocchio dove la qualità satura mentre il costo cresce linearmente in T. Alto valore d'esame (operazionalizza il limite del sampling lento).
 - E4 (regime-shift robustness) — pianificato. Train su un periodo "normale", test su una porzione fuori distribuzione; misura quale modello mantiene gli intervalli onesti sotto shift.
-

7. Discussione

La tesi regge — al contrario di come ci si aspetterebbe. Su Electricity nessun modello addestrato batte il seasonal-naive sul CRPS. Non è un fallimento della pipeline (Exchange lo conferma: là i deep peggiorano come devono su una random walk), ma il finding centrale: su un segnale a forte stagionalità la persistenza stagionale è un avversario durissimo, e la ricchezza distribuzionale della diffusione non ripaga il suo costo di sampling, qui e ora, a questo budget.

La calibrazione è il vero asse di merito, e racconta una storia controintuitiva. Il CRPS da solo (TimeGrad 241.6 < DeepAR 253.7) suggerirebbe che la diffusione "vince tra i deep". Ma la coverage smentisce la sicurezza: TimeGrad è sovra-confidente (cov90 0.647), mentre DeepAR è il meglio calibrato dell'intera scala (cov90 0.831). Per chi deve fidarsi di un intervallo — ad esempio per dimensionare una riserva — DeepAR è preferibile. Il messaggio: un CRPS migliore non implica una calibrazione migliore, e la calibrazione è ciò che un decisore consuma.

La parametrizzazione è una tensione, non uno switch (§6.4). L'arco $x_0 \rightarrow \epsilon$ di TimeDiff è il risultato più istruttivo: due target "corretti da manuale" producono i due fallimenti opposti (collasso vs esplosione della varianza). La lezione generale — una rete non-autoregressiva che denoising un blocco intero fatica a ricomporre la varianza iniettata — è esattamente il tipo di comprensione che il corso premia.

Costo: M3 vs M4. TimeDiff campiona $\sim 6\times$ più veloce di TimeGrad ($\sim 41\text{min}$ vs $\sim 4\text{h } 19\text{min}$) e ha l'errore puntuale più basso tra i deep (MAE 288.5), ma la sua predittiva collassa: velocità senza calibrazione. È il trade-off architetturale autoregressivo↔non-autoregressivo reso concreto.

8. Limiti e minacce alla validità

Dichiariamo i limiti esplicitamente: è parte della tesi di onestà.

- Un solo seed (42). Il protocollo aspirava a ≥ 3 seed con media $\pm$ std; abbiamo bancato un seed per dataset/modello per vincoli di costo (i run deep sono "spedizioni" da ore). Di conseguenza non riportiamo barre d'errore e non dichiariamo vincitori per differenze piccole. È il limite più importante da sanare.
- Il nostro split $\neq$ benchmark pubblicato. I numeri Electricity headline girano sul file grezzo LSTNet (321 serie, split 70/10/20, finestre sliding), non su `electricity_nips` (370 serie, split ufficiale, 7 finestre rolling) su cui sono riportati i CRPS-sum di letteratura. Quindi i nostri numeri non sono direttamente confrontabili con i paper. E0 (§6.1) è il gate che chiude questa ambiguità — prep completo, run da eseguire.
- Budget di training modesto. 50 epoche, nessun tuning esteso degli iperparametri, `diff_steps=100`. Non è il protocollo completo dei paper: una parte della distanza dai numeri pubblicati è imputabile al budget, non all'architettura.
- Esperimenti ancora aperti. E2/E3 sono in corso, E4 pianificato, il toy DDPM in lavorazione, E0 da eseguire. Il nucleo (E1 + E6) è completo e bancato.

- Solo incertezza aleatoria. I diffusion model a pesi puntuali non catturano l'incertezza epistémica (§2.1): una diffusione bayesiana/ensemble è lavoro futuro, non realizzato.
- E6 cross-model da completare. La demo economica è su M0; il confronto CRPS-vs-€ tra modelli attende gli array di forecast di M1/M2/M3 (infrastruttura pronta).

9. Conclusioni e lavoro futuro

Abbiamo costruito una pipeline end-to-end, equa per costruzione (data contract), e una scala di cinque modelli — dal seasonal-naive a due diffusion model condizionali — valutata su quattro pilastri. Il messaggio scientifico è deliberatamente non-trionfalista e quindi credibile: su Electricity la diffusione non batte una baseline stagionale forte sul CRPS; il modello meglio calibrato è DeepAR; e l'ablazione $x_0 \leftrightarrow \epsilon$ di TimeDiff svela che la calibrazione di un blocco non-autoregressivo è una tensione di parametrizzazione, non un interruttore. Exchange, come controllo, conferma che la pipeline è onesta (i deep perdono dove devono).

Lavoro futuro, in ordine d'impatto: (1) eseguire E0 su `electricity_nips` e mettere il nostro CRPS-sum accanto alla tabella pubblicata; (2) ≥ 3 seed con barre d'errore; (3) completare E2/E3 (orizzonte e curva qualità-vs-T) e il toy DDPM; (4) calibrazione seria della diffusione (σ_θ appresa o correzione conformal post-hoc); (5) E6 cross-model (CRPS-vs-€) e (6) la direzione epistémica (diffusione bayesiana/ensemble).

10. Riproducibilità

- Sorgente di verità dei numeri: `results/registry.csv` (schema a colonne, append idempotente via `src/eval/registry.py`). Le tabelle di confronto sono generate da `experiments/make_tables.py`, mai scritte a mano.
- Config: un YAML per dataset (`configs/data_exchange.yaml`, `configs/data_electricity.yaml`, `configs/data_electricity_nips.yaml` per E0). Cambiare dataset = cambiare il config.
- Seed: 42 ovunque (`src/utils/seeds.py`).
- Due ambienti. Uno locale leggero (dati, M0/M1, valutazione, plot, LP di E6) e uno Colab/GPU per i modelli deep M2/M3/M4. Lo stack PyTorchTS $\leftrightarrow$ GluonTS (M2/M3) è fragile e pinnato in `requirements.txt` (con `pandas<2.2` per GluonTS 0.13); M4 TimeDiff è torch puro (torch stock di Colab, nessun pin).
- Parametri Electricity (M0 $\rightarrow$ M4, comparabili): `H=168, =24, m=24, S=100, diff_steps=100, epochs=50, seed=42, CHUNK=256`. M3: `GRU, num_cells=64, layers=2, =linear`. M4: `hidden=64, blocks=4, =cosine, mixup=0.5, sample_steps=100, =1.0`; M4 ϵ identico ma `--param eps`.
- Test: `tests/test_data_contract.py` (split/scaling/no-leakage) + `tests/test_economic.py` (correttezza dell'LP batteria) + `tests/test_metrics.py` (CRPS-sum: `eager == reference` GluonTS-style, `streaming == eager`, forecast perfetto $\rightarrow$ 0).
- Come rilanciare:

```
pip install -r requirements.txt
python -m experiments.run_naive --config configs/data_electricity.yaml # M0
```

```
python -m experiments.run_arima --config configs/data_electricity.yaml # M1
python -m experiments.make_tables # tabelle E1
python -m experiments.plot_presentation # figure
python -m experiments.run_economic # E6
pytest -q # test
# M2/M3/M4 ed E0 girano su Colab - vedi notebooks/ e docs/E0_REPRODUCE_GATE.md
```

Appendice A — Glossario delle metriche

(Più basso = meglio, salvo la coverage che premia la vicinanza al nominale.)

Metrica	Significato
MAE / RMSE MASE	errore L1 / L2 medio della media predittiva MAE scalata dalla MAE in-sample del seasonal-naive; <1 batte il naive. Comparabile solo entro lo stesso dataset.
CRPS	qualità dell'intera distribuzione predittiva (stimatore fair da ensemble); generalizza la MAE alle distribuzioni
CRPS-sum	la CRPS multivariata pubblicata (somma sui canali, normalizzata, decili) — scala-libera, non comparabile col CRPS per-posizione
pinball cov50 / cov90	quantile loss media; 2 × pinball CRPS copertura empirica degli intervalli al 50%/90% (target 0.50/0.90)
width50 / width90	ampiezza media di quegli intervalli (a parità di copertura, più stretto = più informativo)
fit_s / predict_s	secondi di training / inferenza+valutazione

Appendice B — Mappa del repository

```
pml-diffusion-tsf/
  REPORT.md # questo documento
  README.md · CONTRIBUTING.md · requirements.txt
  docs/ # piano implementazione (EN+IT); EO_REPRODUCE_GATE.md; EDA_EXCHANGE
  configs/ # un YAML per dataset (+ data_electricity_nips.yaml per E0)
  src/
    data/ # data contract: loader, split temporale, scaling train-only, finest
    models/ # naive (M0), classical (M1), deepar (M2), timegrad (M3), timediff
    eval/ # metrics.py (CRPS/coverage/CRPS-sum), registry.py, economic.py (E6)
    utils/ # seeds, config, shim freq GluonTS
  experiments/ # run_{naive,arima,deepar,timegrad,timediff}.py + make_tables.py +
  notebooks/ # EDA Exchange + notebook Colab GPU (M2/M3/M4 + E0)
  tests/ # data-contract + economic-LP + metrics (CRPS-sum)
  results/ # registry.csv (verità) + tables/ + economic/
  figures/ # plot generati; presentation/ = figure del deck
```

Appendice C — Riferimenti

- Ho, Jain, Abbeel. Denoising Diffusion Probabilistic Models (DDPM). NeurIPS 2020.
- Rasul, Seward, Schuster, Vollgraf. Autoregressive Denoising Diffusion Models for Multivariate Probabilistic Time Series Forecasting (TimeGrad). ICML 2021.
- Shen, Kwok. Non-autoregressive Conditional Diffusion Models for Time Series Generation (TimeDiff). ICML 2023.
- Tashiro, Song, Song, Ermon. CSDI: Conditional Score-based Diffusion Models for Imputation. NeurIPS 2021.
- Salinas, Flunkert, Gasthaus, Januschowski. DeepAR. International Journal of Forecasting, 2020.
- Gneiting, Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation. JASA 2007.
- Gneiting, Katzfuss. Probabilistic Forecasting. Annual Review of Statistics, 2014.
- Zamo, Naveau. Estimation of the Continuous Ranked Probability Score with Limited Information (stimatore fair del CRPS). Mathematical Geosciences, 2018.
- Diffusion Models for Time Series Forecasting (survey), arXiv:2507.14507, 2025.
- Bortolussi, L. Probabilistic Machine Learning — dispense del corso, Università di Trieste (citare per sezione/pagina, non redistribuite).
- Librerie: GluonTS, PyTorchTS, statsforecast (Nixtla), PuLP/CBC.

Questo report documenta lo stato del progetto alla data dell'ultimo run bancario in *results/registry.csv*. I numeri sono reali; gli esperimenti non ancora eseguiti sono dichiarati come tali. L'onestà sperimentale è la tesi, non un dettaglio.